IT3060– Human Computer Interaction

Year 3, Semester 2, 2026



Lab Exercise 05

Campus: Malabe Campus

Name : Amarasekara A D

IT No : it23547674

Activity 01 - Frontend Technology Comparison

1.1 Strengths & Weaknesses Overview

Flutter

Google's user interface library converts to native code on ARM processor and uses Skia/Impeller rendering engine to provide a consistent interface with code reuse of about 95%.

- Strengths: excellent widget-based UI consistency, strong hot-reload developer experience, solid Flutter Web support.

- Weaknesses: Dart has a smaller talent pool compared to JavaScript, while the AI/ML plugin community for Dart (TFLite, ML Kit bindings) is somewhat behind that of React Native..

React Native

The Meta framework is rendered into native components using the JavaScript bridge (or the more recent Fabric/JSI architecture).

- Strengths: mature ecosystem, biggest multiplatform community, excellent support for complicated gesture-based animations (using Reanimated/Skia), React Native Web for a unified web codebase, and the most extensive collection of pre-built Firebase and realtime libraries.
- Weaknesses: bridge/JSI overhead can add latency for very heavy computation unless offloaded to native modules.

Kotlin Multiplatform (KMP)

Business logic is shared between platforms, whereas the UI is fully native (Jetpack Compose for Android & SwiftUI for iOS).

- Strengths: near-native performance and full access to native APIs, strong type safety.
- Weaknesses: UI code is not shared, so two UI implementations are still needed; web support (Compose Multiplatform for Web) is still experimental; steeper learning curve requiring both Kotlin and native iOS knowledge.

Swift / SwiftUI

Apple's native UI framework.

- Strengths: Best possible performance on iOS, best possible integration with Core ML, ARKit, HealthKit, and platform security.
- Weaknesses: Exclusive to iOS – For Android and Web, different codes are required that will incur twice the expense than iOS only; thus, this will be the least appropriate solution.

1.2 Comparison Across Key Criteria

Criteria	Flutter	React Native	Kotlin Multiplatform	Swift/SwiftUI
Development Speed	High	High	Medium	Medium-Low
Code Reusability	Very High (~95%)	High (~90%)	Medium (logic only)	Low (iOS only)
Performance	Near-native	Near-native	Native	Native (best on iOS)
Ecosystem Support	Strong, growing	Very mature	Growing (JetBrains)	Mature, Apple-only

Learning Curve	Moderate (Dart)	Gentle (JS/React)	Steep	Moderate, iOS-only skill
Web Compatibility	Good (Flutter Web)	Good (RN Web)	Experimental	None
AI/ML Integration	Good (TFLite/ML Kit)	Good (TFLite/ML Kit)	Good, per-platform glue	Excellent (Core ML)
Real-time Features	Good	Excellent	Good, manual setup	Excellent (iOS)
Maintenance Cost	Low-Medium	Low-Medium	Medium	High (2 native teams)
Security	Good	Good	Strong (native)	Strong (native)

1.3 Suitability for FitFlow

For FitFlow, there must be only one development team that will be able to deliver to iOS, Android, and the web in an integrated manner, handle complex animations for the AI "Daily Flow" recommendation card and the social feed, build ML capabilities on-device for adaptive workouts, and implement computer vision through camera capture for nutrition tracking – all under the tight time constraints and limited budget typical of startups. Swift is not even considered due to its single-platform nature. The absence of unified UI and still immature web story make KMP a more difficult proposition than is possible within the current timeline. Both Flutter and React Native can work here; but React Native has an edge over Flutter because of being JavaScript-based and having better options for hiring.

1.4 Recommendation

Recommended: React Native (with React Native Web for the web client), using native modules for the performance-critical computer-vision food-recognition path.

- Fastest realistic path to a single codebase covering iOS, Android and web – critical for the Minimum Lovable Product timeline.
- Largest ecosystem of production-ready libraries for Firebase, real-time sockets, and ondevice ML (TensorFlow Lite via react-native-fast-tflite / ML Kit bindings).
- Proven support for complex, gesture-rich animations needed for the AI recommendation card, drag-and-drop workout builder, and social feed.

This hybrid Swift + Kotlin native approach was not considered since it would take two fullyfledged teams and twice as much money to maintain without any real performance gain; however, it is a feasible alternative solution if the skills of the team were Dart-based. **Activity 2: Compare Backend, Database and Authentication Options 2.1 Backend Frameworks**

Criteria	Node.js / Express (NestJS)	Python / FastAPI	Go

Development Speed	High	High	Medium (more boilerplate)
Real-time Capability	Excellent (Socket.io, native WS)	Good (WebSockets, less mature)	Excellent (goroutines)
AI/ML Integration Ease	Good – calls out to ML service	Excellent – native Python ML stack	Limited – mostly external calls
Ecosystem / Community	Very large (npm)	Strong, fast-growing	Smaller for web frameworks
Learning Curve	Gentle for JS teams	Easy	Steep
Scalability	Good (event loop; offload CPU work)	Good (async/ASGI)	Excellent
Team Fit (shares lang. with RN)	Excellent – same JS/TS stack	Requires context switch	Requires context switch

2.2 Database Options

Criteria	PostgreSQL	MongoDB	Firebase (Firestore/RTDB)	DynamoDB
Scalability	Good (vertical + replicas)	Good (sharding)	Excellent (fully managed)	Excellent (AWSmanaged)
Query Performance	Excellent (joins, ACID)	Good (weaker joins)	Good (simple queries only)	Good, needs key design
Real-time Support	Limited (add-ons needed)	Good (Change Streams)	Best-in-class, native	Good (Streams)
Health Data Handling	Strong - relational integrity, auditfriendly	Moderate - flexible but needs governance	Caution - not HIPAAeligible by default	Strong - AWS BAA available
Criteria	**PostgreSQL**	**MongoDB**	**Firebase (Firestore/RTDB)**	**DynamoDB**
Cost	Moderate	Moderate-High at scale	Low to start, usagebased	Pay-per-request

Team / Stack Fit	Easy for SQL teams	Easy for JS/JSON teams	Very easy with RN + Firebase Auth	AWS-centric teams

2.3 Authentication & Authorization

Criteria	Firebase Auth	AWS Cognito	Auth0	Supabase Auth
Security / Compliance	Good, Google-managed, MFA	Strong; HIPAAeligible with BAA	Excellent; enterprise-grade, SOC2	Good; Postgres row-level security
Integration Ease	Very easy with RN + Firestore	Heavier setup	Easy via SDKs	Easy if using Supabase/Postgres
Social / SSO Providers	Broad (Google, Apple, FB, etc.)	Good (SAML/OIDC)	Extensive, enterprise SSO	Decent
Cost at FitFlow's Scale	Low (generous free tier)	Cost-effective at scale	Higher, scales per user	Low
Customization	Moderate	High (Lambda triggers)	Very high	Good

Conclusion: Firebase Auth is recommended for the quickest and closest integration with the real-time Firebase/React Native stack. Since Firebase by itself is not HIPAA-compliant, FitFlow must add further protections such as Firestore security settings, field-level encryption, and regulated health information stored only in the protected PostgreSQL database to meet the GDPR/CCPA requirements presented in the case study.

Activity 3: Prepare a Technology Comparison Matrix

Each option is scored 1 (poor) - 5 (excellent) per criterion; weighted score = score * weight.

3.1 Frontend Decision Matrix

Criteria (Weight)	Flutter	React Native	Kotlin MP	Swift/SwiftUI
Performance (15%)	4	4	5	5
Ecosystem (10%)	4	5	3	2
Dev. Speed (20%)	5	5	3	2
Security (10%)	4	4	5	5
Cost/Maintainability (15%)	4	4	3	2
AI/ML Support (15%)	3	4	3	5

Cross-platform Reach (15%)	4	5	2	1
Weighted Total	4.05	4.45	3.35	3.05

3.2 Backend + Database + Auth Stack Decision Matrix

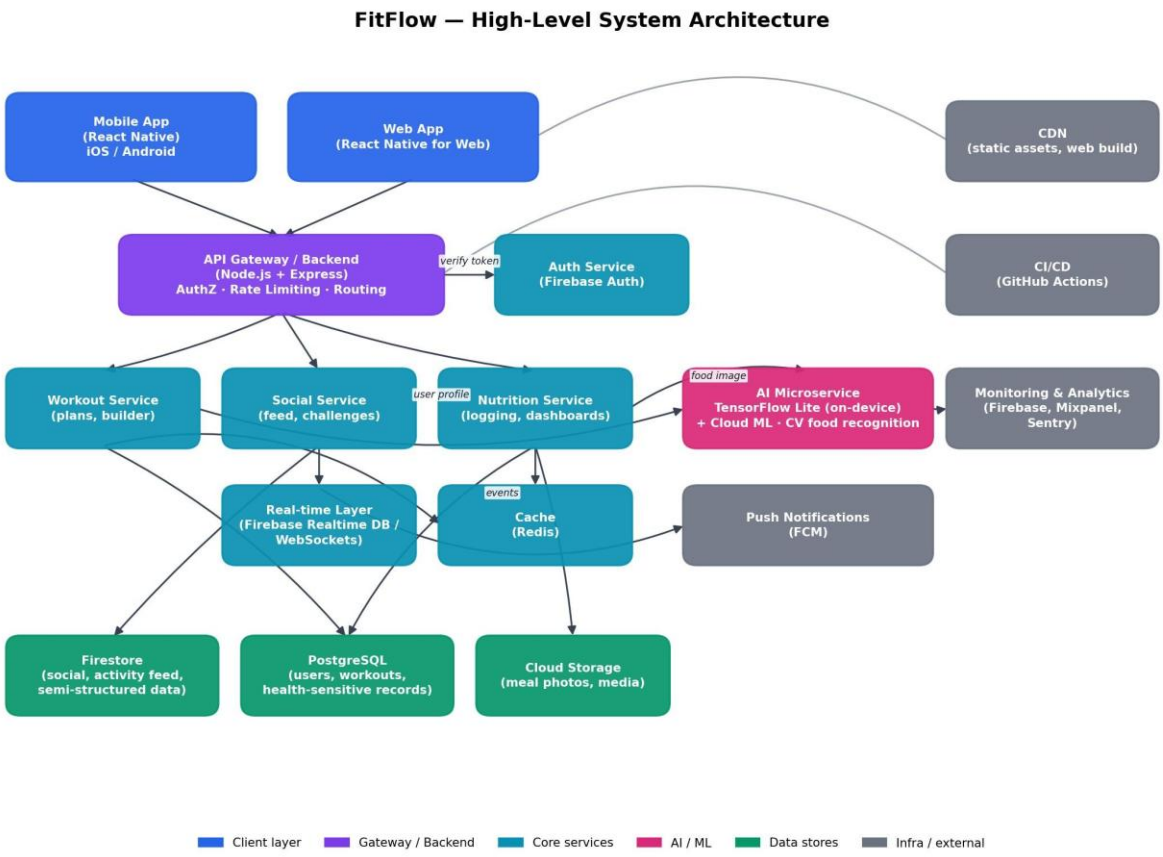
Criteria (Weight)	A: Node+Firebase	B: FastAPI+PG+Auth0	C: Go+DynamoDB+Cognito	D: Hybrid (Recommended)
Performance (15%)	4	4	5	4
Scalability (15%)	5	4	5	5
Dev. Speed (20%)	5	4	2	5
Security/Compliance (20%)	3	5	5	4
Cost (10%)	4	3	4	4
AI/ML Support (10%)	4	5	2	4
Maintainability (10%)	4	4	3	4
Weighted Total	4.15	4.20	3.80	4.35

3.3 Recommended Technology Stack

- Frontend: React Native (+ React Native Web) - 4.45/5
- Backend: Node.js / Express
- Databases: Firestore/Realtime Database (social, real-time) + PostgreSQL (core, healthsensitive)
- Authentication: Firebase Auth, hardened with Firestore security rules and field-level encryption
- AI: TensorFlow Lite (on-device) + cloud ML service for heavier models and computer vision

Activity 4: Design a High-Level Architecture

Below is an illustration of the architecture that follows the suggested stack design, including a client layer, API gateway or backend layer, core domain microservices, artificial intelligence microservices, real-time layer, caching, and hybrid data layer, using CI/CD, monitoring, and CDN.



4.1 Key Components

•Client layer: React Native mobile app (iOS/Android) and React Native for Web, sharing UI code and business logic.

•API Gateway / Backend: Node.js + Express, handling authentication checks, rate limiting and routing to domain services.

•Core services: Workout Service (AI-generated plans, drag-and-drop builder), Social Service (feed, challenges, private circles), Nutrition Service (camera-based logging, dashboards).

•AI Microservice: TensorFlow Lite for on-device personalization plus a cloud ML service for heavier models; computer vision for food recognition uses accessible ML Kit-style pipelines, isolated from the main API for independent scaling.

•Real-time layer: Firebase Realtime Database / WebSockets powering the live social feed and challenge updates, feeding Firebase Cloud Messaging for push notifications.

•Caching: Redis, reducing repeated load on Firestore/PostgreSQL for frequently accessed workout and nutrition data.

•Data layer: Firestore for social/activity data, PostgreSQL for users, workouts and healthsensitive records, Cloud Storage for meal photos and media.

•Supporting infrastructure: GitHub Actions for CI/CD, Firebase/Mixpanel/Sentry for monitoring and analytics, a CDN for static web assets.

4.2 Data Flows for Critical Features

Personalized Workout Plans

Mobile/Web client → API Gateway → Workout Service reads the user profile from PostgreSQL and forwards it to the AI Microservice → on-device TFLite model (or cloud model for complex cases) returns an adaptive plan → Workout Service caches it in Redis and returns it to the client, which renders it on the “Daily Flow” card.

Social Sharing

Client posts an update via the Social Service → written to Firestore → propagated through the Real-time Layer to subscribed clients (private circle members) → Push Notifications (FCM) alert relevant users, keeping the flow low-latency and privacy-scoped to each circle.

Nutrition Tracking

Client captures a meal photo → Nutrition Service uploads the image to Cloud Storage and forwards it to the AI Microservice's computer-vision pipeline → recognized food items and estimated nutrition values are returned → Nutrition Service writes the structured log to PostgreSQL (health-adjacent data) and updates the dashboard, replacing FitFlow's previously tedious manual entry.

4.3 Security, Scalability & Integration Considerations

- Security: All communications using TLS; access controlled by Firestore security rules as well as row and field level access controls on PostgreSQL; health-related fields are encrypted and logged for audit purposes per GDPR/CCPA compliance; Firebase Auth provides short-lived tokens via API Gateway.
- Scalability: Stateless Node.js microservices scale horizontally at the gateway; Firestore and Redis handle real-time/Read heavy traffic; AI microservice scales independently as inference is the most resource-consuming task.
- Integration: The AI Microservice exposes a narrow internal API so backend and AI teams can iterate independently; CI/CD via GitHub Actions runs tests and deploys each service (frontend, backend, ai-service) on its own pipeline, matching the repository structure in Activity 5.

4.4 Architecture Decision Record (ADR-001)

Field	Detail
Title	ADR-001: Technology Stack & Architecture for the FitFlow Redesign
Status	Accepted
Context	FitFlow lost users due to a personalization gap, social isolation and tedious manual nutrition logging (68% onboarding drop-off). The redesign must ship AI-personalized plans, private social circles and camera-based nutrition tracking across iOS, Android and web, by a midsized team, under GDPR/CCPA constraints, on a startup timeline.

Decision	Adopt React Native (+ React Native Web) for the client; Node.js/Express as the API gateway/backend; a hybrid Firestore (realtime/social) + PostgreSQL (core/health-sensitive) data layer; Firebase Auth hardened with security rules and encryption; a dedicated TensorFlow Lite + cloud AI microservice; Redis caching; and GitHub Actions CI/CD.
Consequences (Positive)	Single JS/TS codebase across platforms and most of the stack; fastest realistic path to the Minimum Lovable Product; native real-time support pairs directly with the social/community features; independent scaling of the AI workload.
Consequences (Trade-offs)	Firebase alone is not HIPAA-eligible by default, requiring the added PostgreSQL/encryption layer; the JS bridge in React Native requires native modules for the heaviest computer-vision work; running both a Node service and a Python AI service adds modest operational/polyglot complexity.

Activity 5: Create a GitHub Repository

Repo Link - https://github.com/helloanupa/fitflow-redesign.git